RESEARCH ARTICLE / COMPUTER VISION

AutoDrive : Real-Time Autonomous Driving System Using Computer Vision and Deep Learning

Harsh Bhardwaj	Shrishti Singh	Rajat Kandpal
bhardwajharsh0312@gmail.com	shrisht2723@gmail.com	rajatkandpal12@gmail.com

AutoDrive Team

ABSTRACT

AutoDrive integrates road-object detection, lane segmentation, and camera-based steering-angle regression in a modular visual replay system. Its perception branches use a ten-class YOLO11s detector and a YOLO11s-seg model supervised with lane ribbons derived from TuSimple polylines; steering is estimated by a TensorFlow DAVE-2-style convolutional regressor. Project-reported object detection on 525 held-out COCO-10 images reaches 0.672 AP50 and 0.471 AP50-95. Lane validation on 726 frames yields 0.877-0.890 mask AP50 and 0.363-0.389 mask AP50-95 against the converted ribbon labels. A frame-aligned audit of the saved steering checkpoint on a systematic sample of 505 validation frames measures 22.96° MAE, 33.24° RMSE, and R^2 = -0.8769. Its +21.00° mean error and near-zero correlation identify substantial checkpoint limitations. The architecture, data conversions, component results, and validation limits are documented together to support reproducible analysis. The implemented system provides offline inference and visualization; closed-loop control and vehicle actuation have not been evaluated.

Keywords: autonomous-driving research; behavior cloning; steering-angle regression; YOLO11; lane segmentation; validation.

1. Introduction

Automated-driving systems combine scene understanding with decisions about vehicle motion. Modular methods expose intermediate representations such as object boxes and lane geometry, while learned steering models map camera observations directly to a driving target [1], [5]. Early demonstration-learning systems established the feasibility of steering prediction from visual input [2], [3], [4], but accurate open-loop predictions do not by themselves establish safe closed-loop behavior.

This study examines AutoDrive, a three-branch prototype combining object detection, lane segmentation, and steering regression. Its runtime evaluates the branches independently and displays their outputs in an OpenCV heads-up display (HUD). The study addresses component performance and the gap between a visually integrated demonstration and reliable steering. This distinction matters because behavior-cloning models can fail under covariate shift and provide limited explanations for individual predictions [10], [14], [15].

The contributions are: (i) a code-aligned description of the implemented replay architecture; (ii) documentation of COCO category selection and TuSimple centerline-to-ribbon conversion; (iii) component results and qualitative examples from the project records; and (iv) a frame-aligned audit of the steering checkpoint on a deterministic validation sample. The system is evaluated as an engineering research baseline.

2. Related Work

Learning-based steering evolved from ALVINN and DAVE to convolutional end-to-end models such as DAVE-2 [2], [3], [4]. Conditional imitation and video-based driving models extend the direct-regression formulation [12], [13]. Dataset shift and compounding errors remain central evaluation concerns [14], [15].

YOLO-family detectors combine efficient one-stage inference with a common training interface [6], [7], [8], [9]. Related perception methods include two-stage detection and instance segmentation [16], [17], [18]. Modern detectors use geometric and distribution-aware localization losses [19], [20]. Lane methods use spatial context, instance segmentation, attention distillation, row classification, and anchor attention [21], [22], [23], [24], [25].

TuSimple evaluates lane points at prescribed image rows [11]. The present lane experiment instead converts polylines into masks and evaluates those masks with IoU-based metrics. Its mask scores therefore measure agreement with the generated ribbons and cannot be interpreted as official TuSimple leaderboard scores.

3. System Architecture

Figure 1 places the three inference branches within the implemented runtime. A road frame is resized and submitted to asynchronous object and lane workers and a TensorFlow steering worker. Cached results are drawn into the visual HUD. TensorFlow and OpenCV support the steering graph and visualization [28], [29]; the project source is publicly accessible [35].

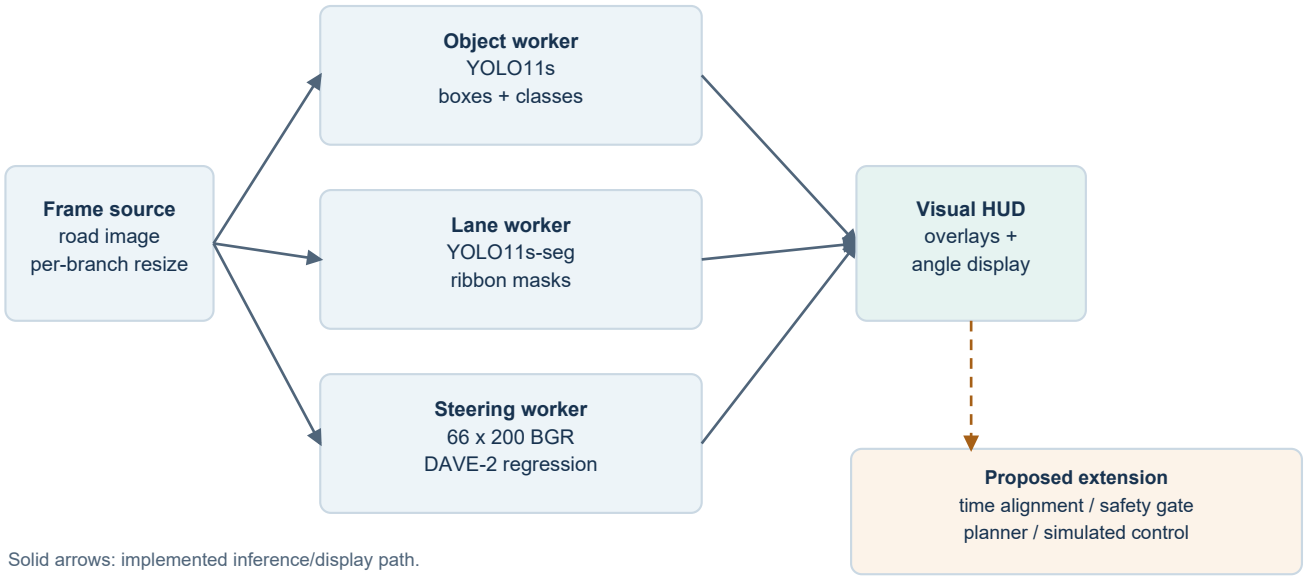


Figure 1. *Implemented inference and visualization path. Object boxes, lane masks, and the steering estimate are displayed together. The dashed branch denotes proposed timestamp alignment, safety supervision, and simulated control, which are not part of the evaluated implementation.*

The current code does not use object or lane outputs to correct the steering estimate, and it provides no validated planner or vehicle-actuation interface. Independent workers keep model calls off the display loop, but cached detections are not associated with source-frame timestamps. Result age and inter-branch alignment therefore remain architectural limitations.

4. Data and Preprocessing

Object detection and lane segmentation use separate public datasets with separate partitions. Steering uses the local indexed driving dataset. Table 1 distinguishes the reported perception splits from the locally checked steering index; no common multimodal test split is available across all three tasks.

Table 1. Datasets, partitions, and annotation targets.

Task	Partitions	Targets
Object detection	COCO-10: 8,080 training; 975 validation; 525 test images	10 classes; 56,267 / 5,604 / 3,116 instances
Lane segmentation	TuSimple: 2,900 training; 726 validation frames	One lane class; 2,732 validation lane instances
Steering regression	45,406 indexed frames: first 36,324 training; final 9,082 validation	Degree-valued angles; ordered, unshuffled partition

4.1 Object labels and class selection

COCO-10 selects person, bicycle, car, motorcycle, bus, truck, traffic light, stop sign, dog, and cat from COCO [27]. Project records describe round-robin sampling across the selected categories, 8,000 sampled training images plus 80 background images, and disjoint validation and test subsets from val2017. Category identifiers are remapped to contiguous model classes. Bounding boxes are clipped to image bounds and normalized to YOLO coordinates. Figure 2 shows converted labels on sample images.

4.2 Lane centerline-to-ribbon conversion

TuSimple represents each lane as sparse centerline points at fixed image rows [11]. For each visible lane, the conversion removes near-duplicate points, estimates tangents and unit normals, and offsets the centerline to construct a closed ribbon. The half-width increases with image row to reflect perspective:

$$w(y) = w_0 + w_1 \, y/H; \quad w_0 = 4 \text{ px}, \quad w_1 = 8 \text{ px}, \quad H = 720 \text{ px}.$$

Coordinates are normalized and clipped to [0, 1]. The project reports a 2,900/726 training/validation split with seed 42 and no independent lane test partition. The width choice directly affects mask-IoU scores.

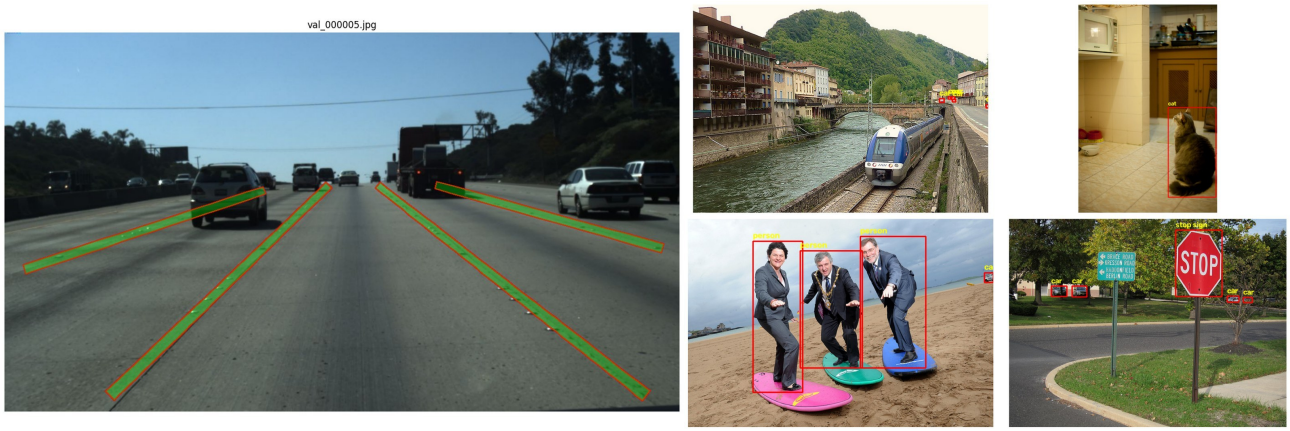


Figure 2. *Label verification examples. Left: TuSimple ribbon polygons over a validation frame (green fill and red outlines). Right: COCO-10 boxes drawn from converted YOLO labels. The examples illustrate the two perception targets.*

Class balancing at image-sampling time does not produce equal object-instance counts because individual images contain different numbers of objects. Figure 3 shows the resulting training-instance imbalance, which must be considered when interpreting per-class performance.



Figure 3. *COCO-10 training-instance distribution on a logarithmic scale. The most and least frequent classes differ by approximately 24:1 in the project records.*

4.3 Steering samples and preprocessing

The local steering index contains 45,406 frame-path/angle pairs. Training labels are converted from degrees to radians. OpenCV loads BGR images; the bottom 150 image rows are retained, resized to 200 × 66 pixels, and divided by 255. The first 80% of index rows form the training subset and the final 20% form validation; the loader does not shuffle the index. The split can therefore reflect temporal or distributional ordering.

The original collection protocol, dataset source, and redistribution license for the steering images could not be established from the available project records. This limits data provenance and independent replication.

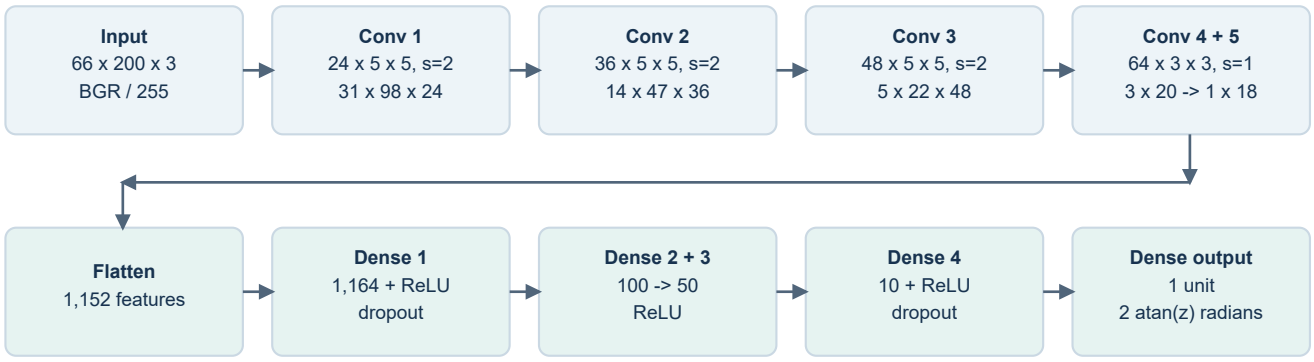
5. Models and Training

5.1 Object detection and lane segmentation

The object branch uses YOLO11s with ten output classes; the lane branch uses YOLO11s-seg with a single lane class [8], [9]. The reported model sizes are 9.42 million and 10.07 million parameters. Both start from vendor-pretrained checkpoints. Object detection is trained for 50 epochs at a 640-pixel input size and batch size 16; lane segmentation is trained for 60 epochs at the same input and batch sizes. The lane configuration uses mosaic = 0 and mask_ratio = 2 to preserve thin structures.

5.2 Implemented steering regressor

The steering graph follows the DAVE-2 convolutional-regression pattern [4] with five valid-padding convolutional layers and five dense layers including its scalar output (Figure 4). ReLU is used in the convolutional layers and four hidden dense layers. Dropout is applied after the 1,164-unit and 10-unit hidden layers. The 66 × 200 × 3 input uses BGR values scaled to [0, 1]. The graph does not perform an RGB-to-YUV conversion or a separate [-1, 1] normalization.



Five convolutional layers and five dense layers, including the output layer.

Figure 4. *Code-aligned steering network. The flattened tensor contains 1,152 features; the first dense layer contains 1,164 units. A scalar affine output z is mapped to 2 atan(z) radians and converted to degrees at inference. Dropout follows the 1,164-unit and 10-unit hidden layers.*

Table 2. Implemented steering graph dimensions and trainable parameters, including biases.

Layer	Configuration and output shape	Parameters
Input	66 × 200 × 3 BGR	-
Conv 1	24 filters, 5 × 5, stride 2; 31 × 98 × 24	1,824
Conv 2	36 filters, 5 × 5, stride 2; 14 × 47 × 36	21,636
Conv 3	48 filters, 5 × 5, stride 2; 5 × 22 × 48	43,248
Conv 4	64 filters, 3 × 3, stride 1; 3 × 20 × 64	27,712
Conv 5	64 filters, 3 × 3, stride 1; 1 × 18 × 64	36,928
Dense layers	1,152 → 1,164 → 100 → 50 → 10 → 1	1,464,163
Total	All trainable weights and biases	1,595,511

5.3 Steering optimization

The training entry point uses Adam [30] with a learning rate of 10^{-4}, batch size 100, 30 nominal epoch loops, and an MSE objective with L2 regularization coefficient 0.001. Dropout keep probability is 0.5 during training and 1.0 during inference. The inspected loader applies no image augmentation.

An implementation detail affects epoch interpretation: the training loop computes batch counts from the full index, while the batch loader cycles only through the first 80% training subset. A nominal loop therefore does not equal one unique pass through the training frames. A validation loader is defined but unused by the training entry point, so validation-based checkpoint selection is not demonstrated.

6. Experimental Design and Metrics

Object detection is evaluated with precision, recall, AP50, AP75, and AP50-95. AP50-95 averages precision over IoU thresholds from 0.50 to 0.95. Lane results include box and mask metrics against the converted ribbons. They do not measure the official TuSimple point-based accuracy or false-positive/false-negative rates.

For steering, let a_i be the recorded angle and p_i the model prediction in degrees, with error $e_i = p_i - a_i$. The audit reports:

$$\mathrm{MAE} = (1/n)\,\Sigma\,|e_i|; \quad \mathrm{RMSE} = \sqrt{[(1/n)\,\Sigma\,e_i^2]}; \quad \mathrm{bias} = (1/n)\,\Sigma\,e_i.$$

$$R^2 = 1 - \Sigma\,e_i^2 \,/\, \Sigma\,(a_i - \mathrm{mean}(a))^2.$$

Pearson correlation and the proportion satisfying $|e_i| \le 5°$ complete the summary. The audit selects every 18th row of the final 9,082-row validation partition, yielding 505 frames. Predictions and labels are compared on the same frame with the preprocessing in Section 4.3, and radian outputs are converted to degrees. This is a systematic validation sample rather than a full held-out test evaluation.

The detection and lane tables and learning curves below are retained from the project experiment records. Original evaluator logs were unavailable for independent re-evaluation. The steering audit is reported separately so that these evidence sources are not treated as one common end-to-end benchmark.

7. Results

7.1 Object detection

The detection learning curves in Figure 5 show rising validation AP and declining losses across 50 epochs. The recorded AP50-95 increases from 0.331 after the first epoch to approximately 0.482 at the end of training, with a reported peak near 0.484. Final-checkpoint curves and best-checkpoint evaluation may differ; Table 3 records the separately reported evaluation values.

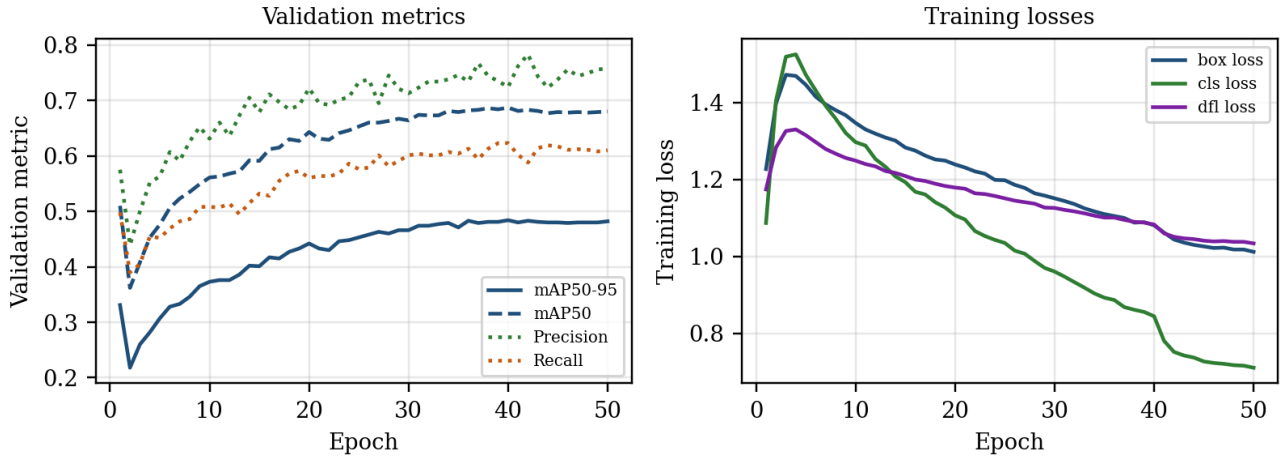


Figure 5. *COCO-10 detector learning curves over 50 epochs. Left: validation AP50-95, AP50, precision, and recall. Right: box, classification, and distribution-focal losses. Curves are preserved from the original project records.*

Table 3. Project-reported object-detection results. The test partition is disjoint from validation.

Split	Images	Instances	P	R	AP50	AP50-95	AP75
Validation	975	5,604	0.724	0.621	0.688	0.486	0.518
Held-out test	525	3,116	0.712	0.614	0.672	0.471	0.511

On the held-out test split, precision (0.712) exceeds recall (0.614), and AP50-95 is 0.471. The validation score is 0.015 higher. Class-level differences are substantial (Table 4 and Figure 6), especially for small categories and classes with few test instances.

Table 4. Per-class held-out test results. N denotes the number of annotated test instances.

Class	P	R	AP50	AP50-95	N
Person	0.812	0.613	0.717	0.457	1,567
Bicycle	0.686	0.399	0.498	0.270	104
Car	0.763	0.607	0.680	0.456	642
Motorcycle	0.802	0.633	0.771	0.472	147
Bus	0.842	0.711	0.791	0.662	114
Truck	0.561	0.463	0.521	0.331	147
Traffic light	0.636	0.476	0.494	0.265	231
Stop sign	0.488	0.607	0.619	0.556	22
Dog	0.651	0.806	0.771	0.596	72
Cat	0.882	0.829	0.861	0.645	70

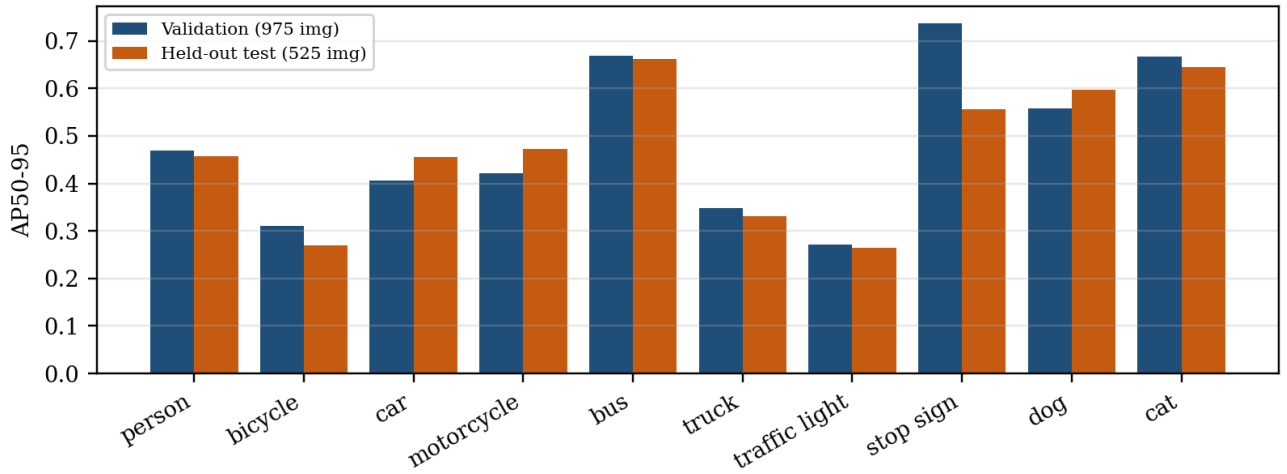


Figure 6. *Per-class AP50-95 on validation (975 images) and held-out test data (525 images). The stop-sign test estimate is based on only 22 instances, so split differences should be interpreted cautiously.*

The strongest reported test classes are bus (AP50-95 = 0.662), cat (0.645), and dog (0.596); traffic light (0.265) and bicycle (0.270) are weaker. Qualitative detections are shown in Figure 7. The examples illustrate model behavior and do not replace the aggregate evaluation.



Figure 7. *Object predictions on six held-out COCO-10 images at confidence ≥ 0.25 and NMS IoU = 0.7. The original examples include large vehicles, partially occluded people, smaller road users, and an animal category.*

7.2 Lane segmentation

Lane training improves more rapidly in box terms than in mask terms (Figure 8). The project records report box AP50-95 increasing from 0.672 to 0.831 over 60 epochs, while mask AP50-95 rises from 0.173 to 0.363. The transient early dip recovers within the initial epochs. The smaller mask score reflects the sensitivity of thin ribbons to lateral displacement and to the chosen width rule.

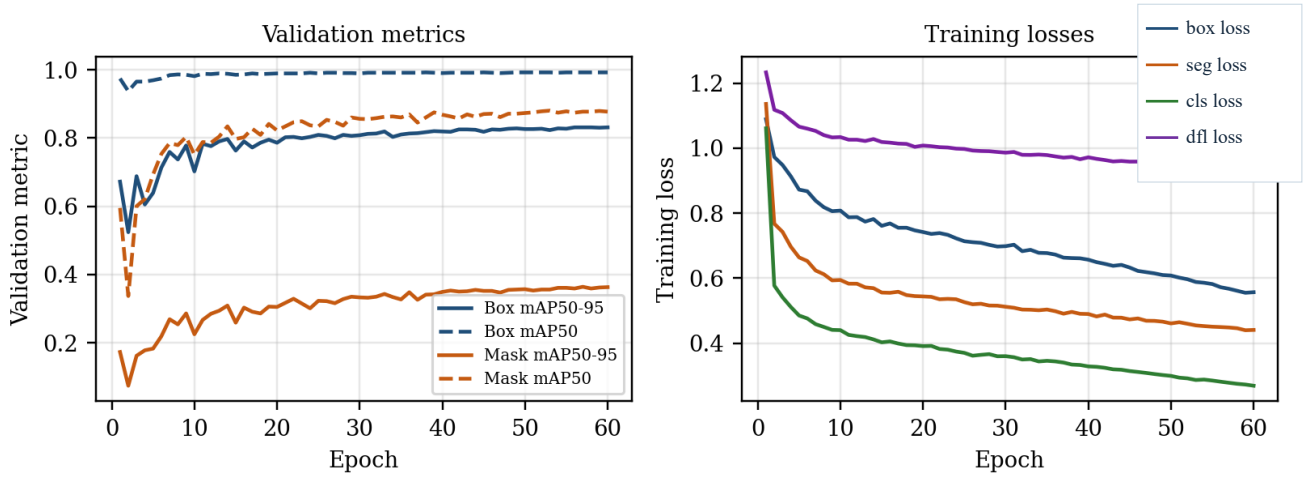


Figure 8. *TuSimple ribbon-segmentation learning curves over 60 epochs. Left: box and mask AP50 and AP50-95. Right: box, segmentation, classification, and distribution-focal losses. Loss terms cover box regression, segmentation, classification, and distribution-focal localization.*

Table 5. Project-reported lane validation on 726 frames and 2,732 lane instances.

Evaluation	Box P	Box R	Box AP50	Box AP50-95	Mask P	Mask R	Mask AP50	Mask AP50-95
End of training	0.976	0.974	0.992	0.831	0.918	0.916	0.877	0.363
Standalone re-evaluation	0.976	0.974	0.992	0.831	0.925	0.923	0.890	0.389

Standalone re-evaluation reports mask AP50 = 0.890 and AP50-95 = 0.389, compared with 0.877 and 0.363 in the training-run validator. Both results are retained because the 0.026 AP50-95 difference indicates evaluation-configuration sensitivity. No independent lane test split is reported. Figure 9 shows the original validation examples under different traffic and shadow conditions.

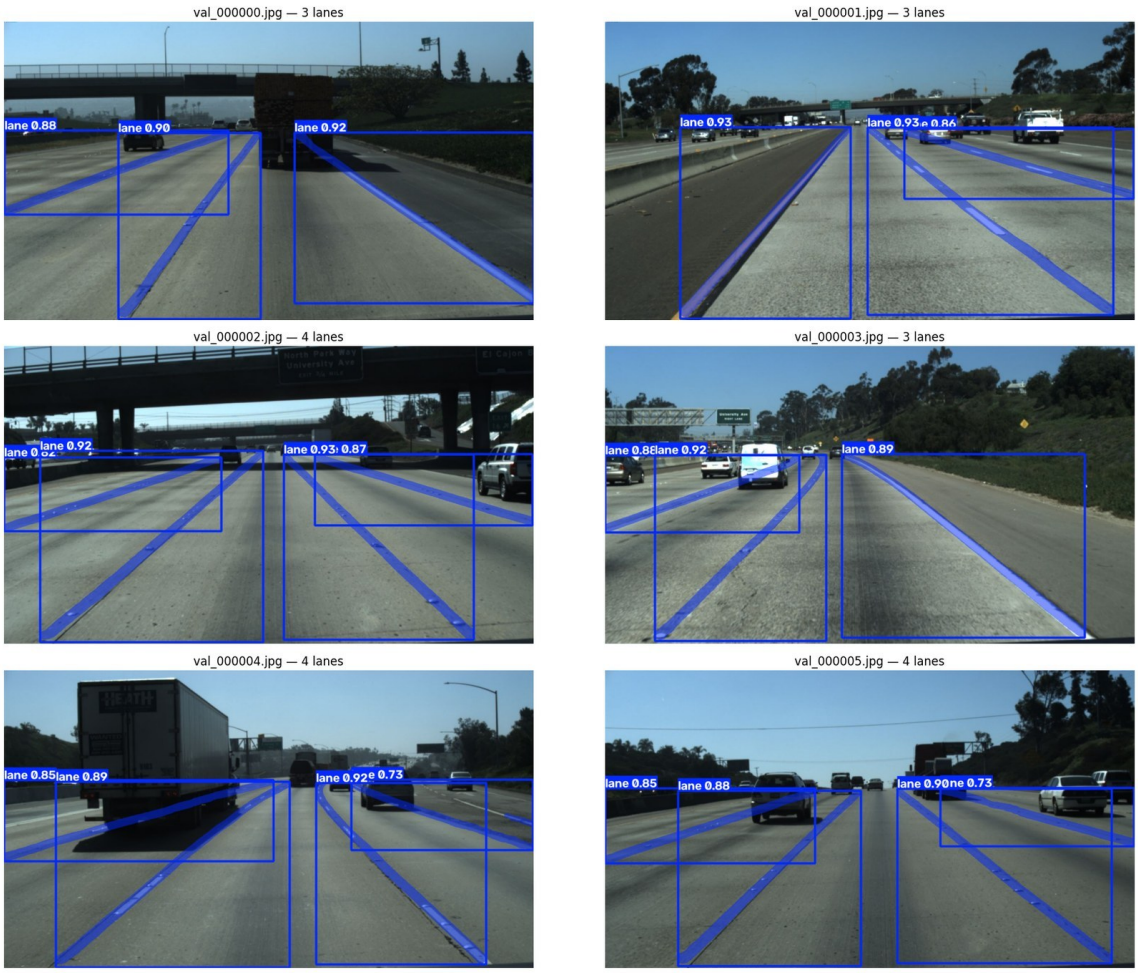


Figure 9. *Lane predictions on six TuSimple validation frames at confidence ≥ 0.25. Three or four lanes are detected per frame in these examples, with reported confidences approximately 0.73-0.93. Large overlapping boxes arise from diagonal, thin lane masks.*

7.3 Confusion diagnostics

Figure 10 places the validation confusion matrices beside the interpretation they support. The lane record contains 2,706 matched lane instances, 119 false-positive lane predictions, and 26 missed lanes. The detector matrix identifies semantic confusion and missed objects at the evaluated confidence threshold; these validation diagnostics use a different split from the held-out detection table.

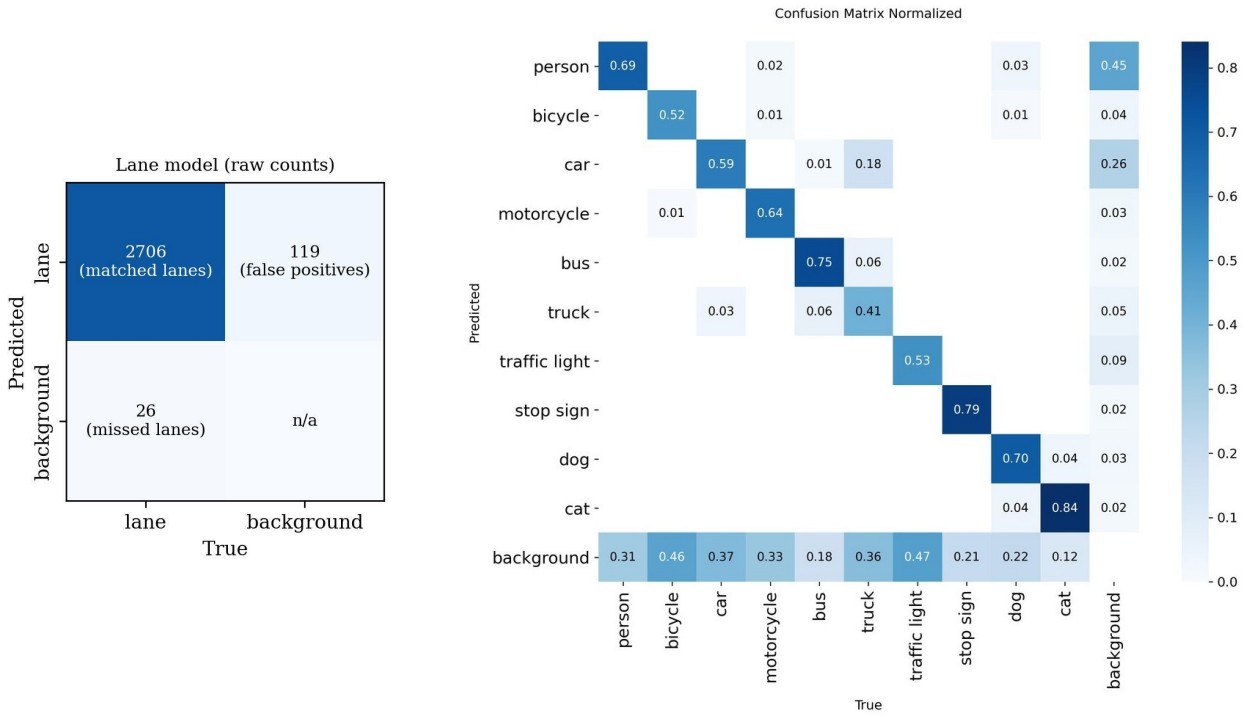


Figure 10. *Validation confusion matrices. Left: lane-model raw counts (columns: true class; rows: predicted class). Right: detection matrix normalized by true class. Its bottom row represents missed objects and its rightmost column represents predictions on background.*

The detector confuses truck with car for approximately 0.18 of true truck instances in the recorded matrix. Missed-object proportions are about 0.31 for person, 0.46 for bicycle, 0.47 for traffic light, and 0.37 for car. These patterns are consistent with lower recall for small or ambiguous categories.

7.4 Steering validation audit

The checkpoint audit compares synchronous model outputs with labels on 505 selected validation frames. Table 6 reports degree-valued errors after output conversion. Negative R^2 and near-zero correlation indicate that this checkpoint does not provide useful steering estimates on the sampled partition.

Table 6. Steering checkpoint audit on the systematic validation sample (n = 505).

Metric	Result	Interpretation
MAE	22.9616°	Mean absolute angular error
RMSE	33.2350°	Root mean squared angular error
Mean signed error	+20.9977°	Strong positive prediction bias
R^2	-0.8769	Worse than predicting the sample mean
Pearson correlation	-0.0404	Near-zero linear association
Within ±5°	9.11%	Share of predictions meeting the tolerance

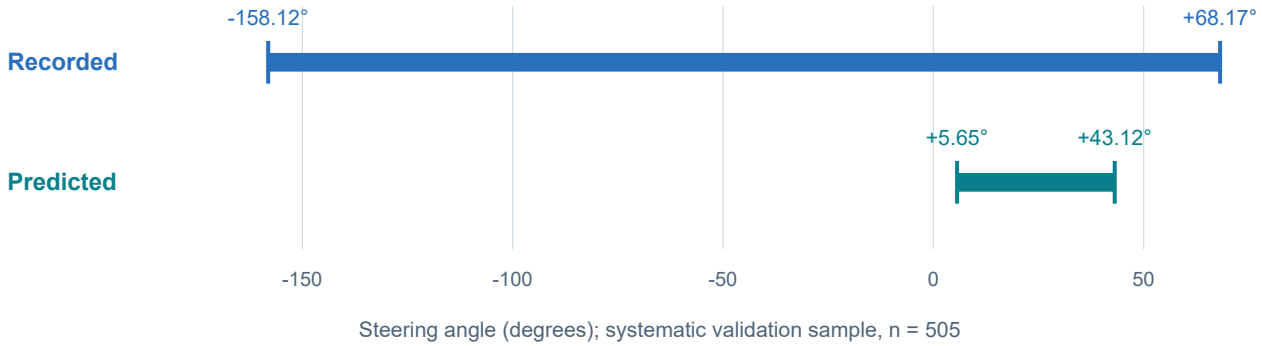


Figure 11. *Recorded and predicted steering-angle ranges in the same 505-frame validation sample. Recorded angles span -158.12° to +68.17°, whereas predictions remain between +5.65° and +43.12°. The range summary illustrates the observed positive bias.*

For example, a recorded -54.86° target is paired with a +16.85° prediction, and a +51.23° target with a +9.21° prediction. The aggregate results and range mismatch support a diagnostic conclusion about the current checkpoint rather than a claim of successful autonomous steering.

7.5 Latency and footprint

The project manuscript records Tesla T4 per-image totals of 11.1 ms for detection and 11.8 ms for lane segmentation under batch-16 timing. Single-image totals are reported as 17.0 ms and 14.3 ms. Adding the two component times gives 22.9-31.3 ms, corresponding to a component-only sequential projection of approximately 32-44 frames/s. This projection excludes frame capture, steering, synchronization, display, and control I/O. It is not an independently reproduced end-to-end runtime benchmark.

8. Discussion and Limitations

The object and lane experiments provide component baselines, while the steering audit reveals a substantial gap between visual demonstration and reliable angle prediction. The sample has negative R^2, effectively zero correlation, and a strong positive bias. The implemented graph contains 1,595,511 trainable parameters with a 1,164-unit first dense layer. Figures 1 and 4 and Table 2 describe that implementation consistently.

Several evaluation limits constrain generalization. The steering sample comes from an ordered validation partition, rather than an independently collected or session-grouped test set. Its data provenance and license are undocumented. Perception results and learning curves are project-reported records whose original evaluator logs were unavailable for independent rerun. Experiments are single-run, with no confidence intervals or systematic ablation study. The lane mask target depends on an engineered ribbon width and is distinct from the official TuSimple target.

No closed-loop simulation or public-road evaluation is reported. Perception outputs are displayed without temporal alignment and do not supervise steering. Consequently, component accuracy and latency figures cannot establish a safe autonomous-driving capability or a complete end-to-end frame rate.

9. Proposed Architecture and Further Evaluation

A stronger next iteration should first rebuild the steering partition by driving session, verify angle sign and degree/radian conventions, introduce a separate test set, and select checkpoints using validation loss. The runtime should attach frame identifiers and timestamps to every result, reject stale outputs, and evaluate a confidence-aware supervisory layer before planning is introduced. The dashed extension in Figure 1 denotes this proposed work rather than an evaluated feature.

Closed-loop evaluation can then be conducted in a simulator such as CARLA [26], with lane departure, collision, and intervention measures in addition to open-loop angular errors. Grad-CAM-style analysis may help inspect steering-network attention [31], while datasets such as BDD100K, KITTI, and nuScenes can support broader scene and domain coverage [32], [33], [34]. These are experimental directions; no such additional results are claimed here.

10. Conclusion

AutoDrive brings object detection, lane segmentation, and steering-angle regression into a modular visual replay system. The reported detection and lane results demonstrate component-level perception performance within their dataset-specific protocols. The steering audit measures 22.96° MAE, 33.24° RMSE, and R^2 = -0.8769 on a systematic sample of 505 validation frames, identifying a checkpoint that requires further development. The code-aligned architecture and explicit evaluation boundaries support a reproducible research account and define the data, training, and closed-loop validation work needed next.

Declarations

Author contributions. Harsh Bhardwaj, Shrishti Singh, and Rajat Kandpal contributed to conceptualization, implementation, validation, analysis, and manuscript preparation. All three authors reviewed and approved the final manuscript.

Funding. This research received no external funding.

Competing interests. The authors declare no competing interests.

Data availability. COCO and TuSimple are publicly documented datasets, subject to their respective access and licensing terms. The steering dataset source, collection protocol, and redistribution terms were not established from the available project records; the steering images are not redistributed with this manuscript.

Code availability. The AutoDrive Team project repository is available at https://github.com/HA2345567/Autodrive. Source files describe training, inference, and visualization; large datasets and model artifacts require separate access. Repository access date: 21 October 2026.

Scope and safety. The reported system is an offline research prototype and has not been validated for vehicle control or public-road operation.

References

[1] SAE International, Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles, SAE Standard J3016_202104, 2021.

[2] D. A. Pomerleau, ALVINN: An autonomous land vehicle in a neural network, in Advances in Neural Information Processing Systems 1, 1989, pp. 305-313.

[3] Y. LeCun, U. Muller, J. Ben, E. Cosatto, and B. Flepp, Off-road obstacle avoidance through end-to-end learning, in Advances in Neural Information Processing Systems 18, 2005.

[4] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, End to end learning for self-driving cars, arXiv:1604.07316, 2016.

[5] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, End-to-end autonomous driving: Challenges and frontiers, IEEE Trans. Pattern Anal. Mach. Intell., 2024 (arXiv:2306.16927).

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, You only look once: Unified, real-time object detection, in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.

[7] J. Terven, D.-M. Cordova-Esparza, and J.-A. Romero-Gonzalez, A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS, Mach. Learn. Knowl. Extr., vol. 5, no. 4, pp. 1680-1716, 2023.

[8] G. Jocher and J. Qiu, Ultralytics YOLO11, 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] R. Khanam and M. Hussain, YOLOv11: An overview of the key architectural enhancements, arXiv:2410.17725, 2024.

[10] M. Bojarski, P. Yeres, A. Choromanska, K. Choromanski, B. Firner, L. Jackel, and U. Muller, Explaining how a deep neural network trained with end-to-end learning steers a car, arXiv:1704.07911, 2017.

[11] TuSimple, TuSimple lane detection benchmark, 2017. [Online]. Available: https://github.com/TuSimple/tusimple-benchmark

[12] F. Codevilla, M. Muller, A. Lopez, V. Koltun, and A. Dosovitskiy, End-to-end driving via conditional imitation learning, in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2018, pp. 4693-4700.

[13] H. Xu, Y. Gao, F. Yu, and T. Darrell, End-to-end learning of driving models from large-scale video datasets, in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017.

[14] F. Codevilla, E. Santana, A. M. Lopez, and A. Gaidon, Exploring the limitations of behavior cloning for autonomous driving, in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 9329-9338.

[15] S. Ross, G. J. Gordon, and J. A. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in Proc. Int. Conf. Artif. Intell. Stat. (AISTATS), 2011, pp. 627-635.

[16] S. Ren, K. He, R. Girshick, and J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137-1149, 2017.

[17] K. He, G. Gkioxari, P. Dollar, and R. Girshick, Mask R-CNN, in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 2961-2969.

[18] D. Bolya, C. Zhou, F. Xiao, and Y. J. Lee, YOLACT: Real-time instance segmentation, in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 9157-9166.

[19] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, Distance-IoU loss: Faster and better learning for bounding box regression, in Proc. AAAI Conf. Artif. Intell., 2020, pp. 12993-13000.

[20] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection, in Advances in Neural Information Processing Systems 33, 2020.

[21] X. Pan, J. Shi, P. Luo, X. Wang, and X. Tang, Spatial as deep: Spatial CNN for traffic scene understanding, in Proc. AAAI Conf. Artif. Intell., 2018, pp. 7276-7283.

[22] D. Neven, B. De Brabandere, S. Georgoulis, M. Proesmans, and L. Van Gool, Towards end-to-end lane detection: An instance segmentation approach, in Proc. IEEE Intell. Veh. Symp. (IV), 2018, pp. 286-291.

[23] Y. Hou, Z. Ma, C. Liu, and C. C. Loy, Learning lightweight lane detection CNNs by self attention distillation, in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 1013-1021.

[24] Z. Qin, H. Wang, and X. Li, Ultra fast structure-aware deep lane detection, in Proc. Eur. Conf. Comput. Vis. (ECCV), 2020, pp. 276-291.

[25] L. Tabelini, R. Berriel, T. M. Paixao, C. Badue, A. F. De Souza, and T. Oliveira-Santos, Keep your eyes on the lane: Real-time attention-guided lane detection, in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2021, pp. 294-302.

[26] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, CARLA: An open urban driving simulator, in Proc. Conf. Robot Learn. (CoRL), 2017, pp. 1-16.

[27] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, Microsoft COCO: Common objects in context, in Proc. Eur. Conf. Comput. Vis. (ECCV), 2014, pp. 740-755.

[28] M. Abadi et al., TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. [Online]. Available: https://www.tensorflow.org

[29] G. Bradski, The OpenCV library, Dr. Dobbs Journal of Software Tools, 2000.

[30] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.

[31] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, Grad-CAM: Visual explanations from deep networks via gradient-based localization, in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618-626.

[32] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, BDD100K: A diverse driving dataset for heterogeneous multitask learning, in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 2636-2645.

[33] A. Geiger, P. Lenz, and R. Urtasun, Are we ready for autonomous driving? The KITTI vision benchmark suite, in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2012, pp. 3354-3361.

[34] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, nuScenes: A multimodal dataset for autonomous driving, in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 11621-11631.

[35] AutoDrive Team, "AutoDrive: Real-Time Autonomous Driving System Using Computer Vision and Deep Learning," GitHub repository. Available: https://github.com/HA2345567/Autodrive. Accessed: Oct. 21, 2026.